

SceneGuard: AI-Powered Production Feasibility Analysis

Hackathon Project | Problem Statement PS-3

1. Problem Statement

Film productions fail when scenes that look compelling on paper turn out to be high-risk, costly, or impossible to execute. Pre-production teams face a critical challenge:

How do you know if a scene is feasible before committing resources?

Real-world production constraints include:

- **Budget overruns** from underestimated complexity
- **Safety risks** in stunts, crowds, or hazardous environments
- **Logistics nightmares** with location access, permits, equipment
- **Technical challenges** in VFX, lighting, special effects
- **Weather dependencies** that can halt outdoor shoots

Currently, producers rely on:

- Manual script breakdowns (time-consuming, subjective)
- Experience-based gut feelings (inconsistent across teams)
- Late-stage discoveries (expensive to fix during production)

The Gap: No systematic, AI-assisted way to evaluate scene feasibility early in pre-production with reasoning-based, explainable risk assessment.

2. Why Existing AI Chatbots Are Not Enough

Directly prompting Gemini or ChatGPT with "Is this scene feasible?" has critical limitations:

Unstructured Output

- No standardized format
- Different answers each time
- Hard to compare scenes
- Can't feed into production pipelines

Non-Deterministic Answers

- Same scene → different risk levels on re-analysis
- No reproducibility
- Can't trust for decision-making

No Production Checklist Mapping

- Generic advice, not production-specific
- Doesn't map to real pre-production phases
- Missing crew requirements, location needs, safety protocols

No Reasoning Transparency

- "Black box" recommendations
- Can't explain why a scene is high-risk
- No evidence trail for stakeholders

No Multi-Dimensional Analysis

- Doesn't separate Safety vs. Budget vs. Logistics
- No constraint interaction understanding
- Missing weather feasibility for outdoor scenes

SceneGuard addresses all of these limitations.

3. SceneGuard: Our Solution

SceneGuard is a **reasoning-based production feasibility system** that analyzes film scenes through multiple specialized lenses:

What it does:

- Extracts production-relevant facts from scene descriptions
- Evaluates feasibility across Safety, Logistics, Budget, Technical dimensions
- Assesses weather sensitivity for outdoor scenes
- Generates production checklists and mitigation strategies
- Provides explainable, consistent risk scoring

Key Innovation:

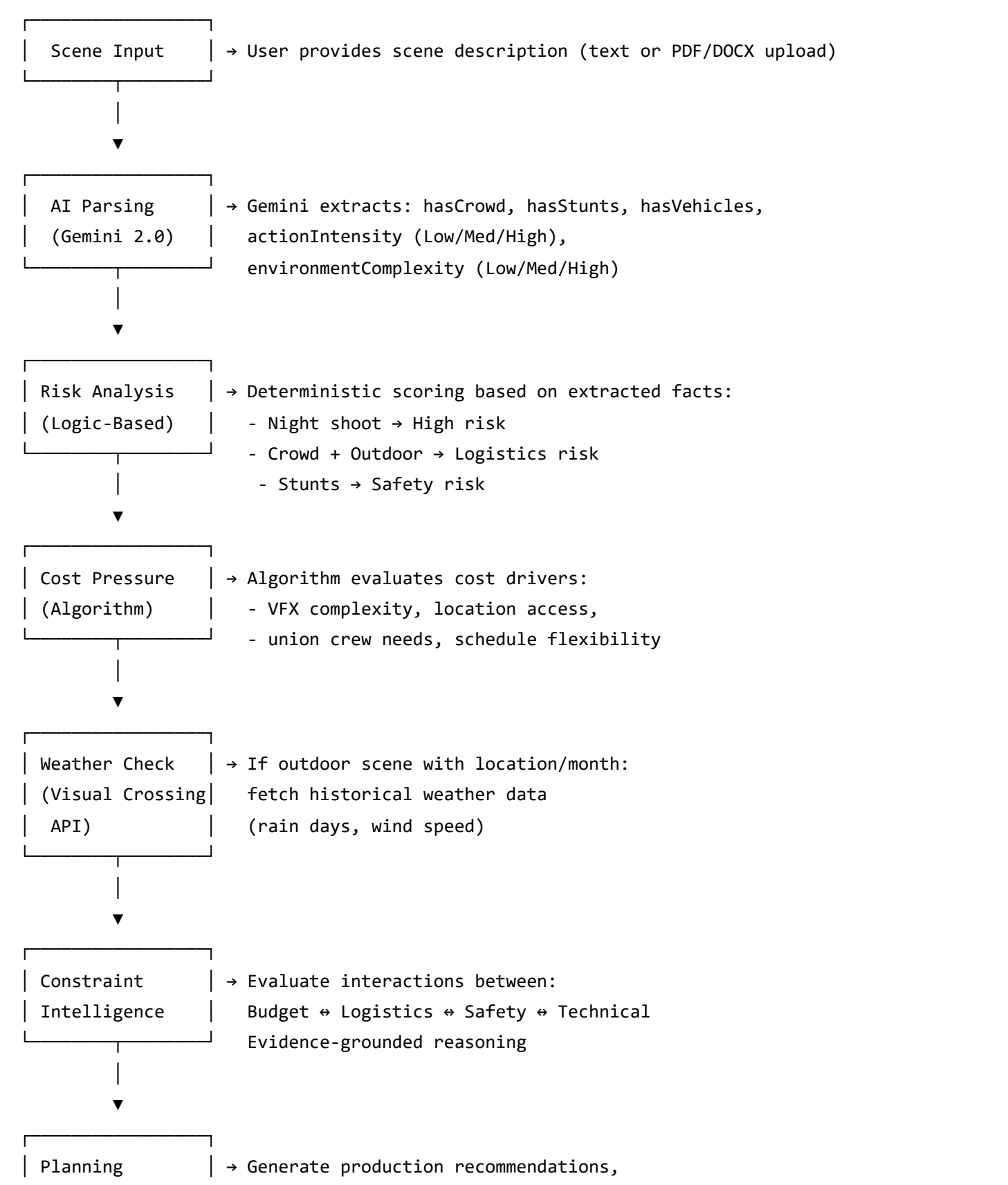
Instead of one-shot AI generation, SceneGuard uses a **multi-stage reasoning pipeline**:

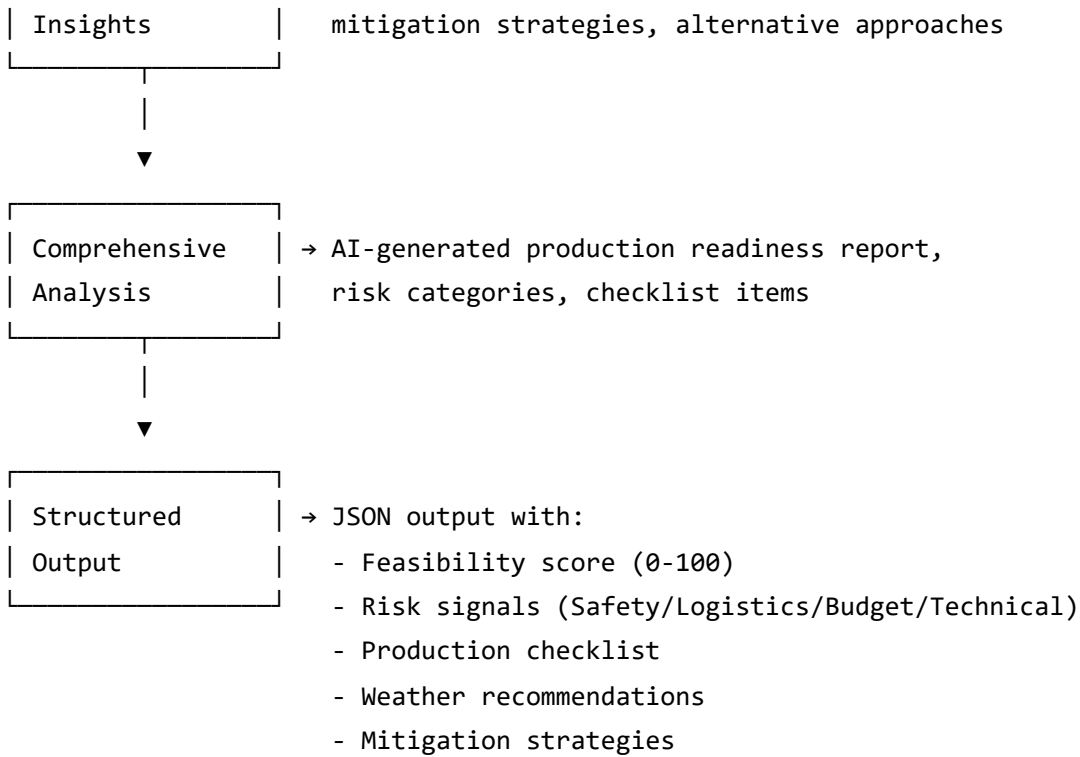
1. AI extracts structured facts (has crowd? has stunts? complexity level?)
2. Deterministic algorithms reason over those facts
3. Production-aligned constraints are evaluated
4. Explainable outputs are generated

Result: Consistent, defensible feasibility analysis that production teams can trust.

4. How SceneGuard Works (Step-by-Step)

End-to-End Pipeline





Actual Implementation Flow

Step 1: Scene Input

- User provides description via text input or file upload (PDF/DOCX)
- File parser extracts text using pdf-parse library
- Metadata captured: category (Indoor/Outdoor/VFX), time of day, location, month

Step 2: AI Fact Extraction (Gemini 2.0 Flash)

```
// Structured parsing - NOT free-form generation
{
  hasCrowd: boolean,           // Is there a crowd scene?
  hasStunts: boolean,          // Are stunts involved?
  hasVehicles: boolean,        // Vehicles in scene?
  actionIntensity: 'Low' | 'Medium' | 'High',
  environmentComplexity: 'Low' | 'Medium' | 'High'
}
```

Step 3: Risk Signal Generation (Deterministic Logic)

```
// Algorithm-based, NOT AI-generated
if (hasStunts) → Safety: High
if (hasCrowd && sceneCategory === 'Outdoor') → Logistics: High
if (timeOfDay === 'Night') → Technical: High
if (hasVehicles) → Budget: Medium
```

Step 4: Cost Pressure Analysis

```
// Rule-based evaluation
drivers = []
if (actionIntensity === 'High') → drivers.push('High action complexity')
if (environmentComplexity === 'High') → drivers.push('Complex environment setup')
costPressure = (driverCount > 3) ? 'High' : 'Medium'
```

Step 5: Weather Feasibility (External API)

- If outdoor scene with location + month provided
- Query Visual Crossing Weather API for historical data
- Calculate average rain days, wind speed
- Generate recommendation (e.g., "June has 12 rain days - plan weather cover")

Step 6: Constraint Intelligence

- Analyze Budget ↔ Logistics interaction (e.g., remote location = high transport cost)
- Analyze Safety ↔ Technical interaction (e.g., stunts + VFX = coordination complexity)
- Generate evidence map showing WHY constraints exist

Step 7: Planning Insights

- AI-generated but structured recommendations
- Mitigation strategies specific to identified risks
- Alternative approaches (e.g., "Consider studio setup instead of outdoor location")

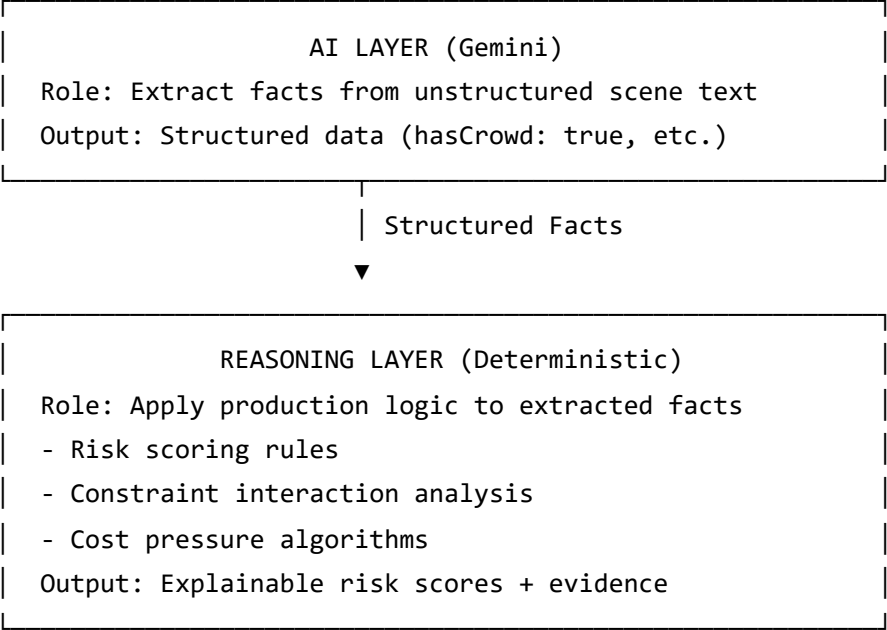
Step 8: Structured Output

- All results packaged in consistent JSON schema
- Stored in PostgreSQL database for history tracking
- Displayed in production-ready dashboard

5. Reasoning-Based Evaluation (Core Innovation)

The Two-Stage Approach

SceneGuard separates **extraction** from **reasoning**:



Example: Night Shoot Risk Analysis

Input Scene: "A chase sequence through a dark alley at midnight"

Stage 1: AI Extraction

```
{
  "timeOfDay": "Night",
  "actionIntensity": "High",
  "environmentComplexity": "Medium"
}
```

Stage 2: Deterministic Reasoning

```
// Risk Analyzer Logic (lib/risk-analyzer.ts)
const signals = []

if (timeOfDay === 'Night') {
  signals.push({
    name: 'Night Shoot',
    level: 'High',
    reason: 'Requires specialized lighting equipment, extended setup time, premium crew rates'
  })
}

if (actionIntensity === 'High') {
  signals.push({
    name: 'Complex Action',
    level: 'High',
    reason: 'Coordination challenges, safety protocols, multiple takes'
  })
}

// Constraint Interaction
if (isNightShoot && hasHighAction) {
  interactionRisk = 'High'
  reasoning = 'Night lighting + fast-paced action = difficult coverage, safety concerns'
}
```

Why This Matters:

- **Reproducible:** Same facts → same risk score, every time
- **Explainable:** Each risk signal traces back to specific scene elements
- **Auditable:** Production teams can verify the logic
- **Defensible:** Stakeholders understand WHY a scene is high-risk

6. Risk Categories Explained

SceneGuard evaluates four production dimensions:

Category	What It Measures	Example Triggers	Risk Indicators
Safety	Physical hazards to cast/crew	Stunts, heights, water, fire, crowds	Requires safety coordinator, insurance coverage, stunt coordinator
Logistics	Coordination & execution complexity	Remote locations, crowd control, permits, equipment transport	Location scouting, access agreements, traffic management
Technical	Creative/technical execution difficulty	VFX, special effects, night shoots, complex lighting	Specialized crew, extended setup time, post-production requirements
Budget	Cost pressure & resource intensity	Union crew, controlled sets, extras, equipment rentals	Line producer review, budget contingency planning

Risk Signal Generation Logic

Scene Element	→	Risk Category	→	Evidence
hasStunts = true	→	Safety: High	→	"Scene involves physical stunts requiring safety coordination"
hasCrowd = true	→	Logistics: High	→	"Crowd management requires permits, security, coordination"
timeOfDay = Night	→	Technical: High	→	"Night shoots require extensive lighting, premium equipment"
environmentComplexity = High	→	Budget: Medium	→	"Complex environments increase setup and equipment costs"

Constraint Interaction Matrix

SceneGuard analyzes how constraints interact:

	Budget	Logistics	Safety	Technical
Budget	–	Medium	Low	Medium
Logistics	Medium	–	High	Medium
Safety	Low	High	–	High
Technical	Medium	Medium	High	–

Example Interaction:

- **Safety ↔ Logistics (High):** Stunt scenes require safety zones, which affect crowd placement and location access
- **Technical ↔ Budget (Medium):** VFX-heavy scenes need specialized crew (higher day rates) and extended post-production

7. Feasibility & Cost Interpretation

Feasibility Scoring (0-100)

SceneGuard calculates a composite feasibility score:

```
Feasibility = Base Score (100)
- High Risk Penalty (each High risk: -15 points)
- Medium Risk Penalty (each Medium risk: -8 points)
- Constraint Interaction Penalty (critical interactions: -10 points)
+ Mitigation Bonus (if clear alternatives exist: +5 points)
```

Interpretation Scale:

85-100		Low Risk	Straightforward execution
65-84		Medium Risk	Manageable with planning
45-64		High Risk	Requires extensive mitigation
0-44		Very High Risk	Consider alternatives

Cost Pressure (Relative, Not Absolute)

SceneGuard does **NOT** predict exact budgets. Instead, it identifies **cost pressure drivers**:

Low Pressure:

- Simple indoor dialogue
- Minimal crew requirements
- Standard equipment

Medium Pressure:

- Moderate complexity
- Some specialized needs

- Manageable logistics

High Pressure:

- Multiple high-complexity elements
- Specialized crew + equipment
- Extended setup/breakdown time

Example Output:

Cost Pressure: High

Drivers:

- ✓ Night shoot (premium crew rates)
- ✓ Complex lighting setup (equipment rental + time)
- ✓ Outdoor location (transport, permits, weather contingency)
- ✓ Action sequence (safety crew, multiple takes, stunt coordination)

Explanation: This scene combines multiple cost-intensive elements. Budget should include contingency for weather delays and extended night shoot hours.

8. Production Checklist Mapping

SceneGuard generates production-phase checklists based on scene analysis:

Example: High-Risk Action Scene

PRE-PRODUCTION PHASE
☐ Hire Stunt Coordinator ↳ Reason: Scene has stunts (Safety: High)
☐ Scout and Secure Outdoor Location ↳ Reason: Outdoor shoot (Logistics: High) ↳ Requirements: Access agreement, permits, insurance
☐ Budget for Lighting Package ↳ Reason: Night shoot (Technical: High) ↳ Equipment: HMIs, generators, diffusion

PRODUCTION PHASE
☐ Safety Meeting Before Each Stunt ↳ Attendees: Director, Stunt Coordinator, AD, Medic
☐ Weather Monitoring ↳ Reason: June avg 12 rain days ↳ Contingency: Cover set or reschedule
☐ Extended Setup Time ↳ Reason: Night lighting + action coverage ↳ Schedule: +3 hours for lighting setup

MITIGATION STRATEGIES
• Consider shooting day-for-night instead (reduces lighting costs, crew fatigue)
• Pre-visualize stunt sequence with storyboards (reduces on-set trial time, improves safety)
• Use controlled studio space for portions

(eliminates weather risk, permits)

Mapping to Real Production Workflow

SceneGuard Output → Production Action

Safety: High (Stunts) → Line item: Stunt coordinator
→ Insurance: Stunt coverage add-on
→ Schedule: Safety rehearsals

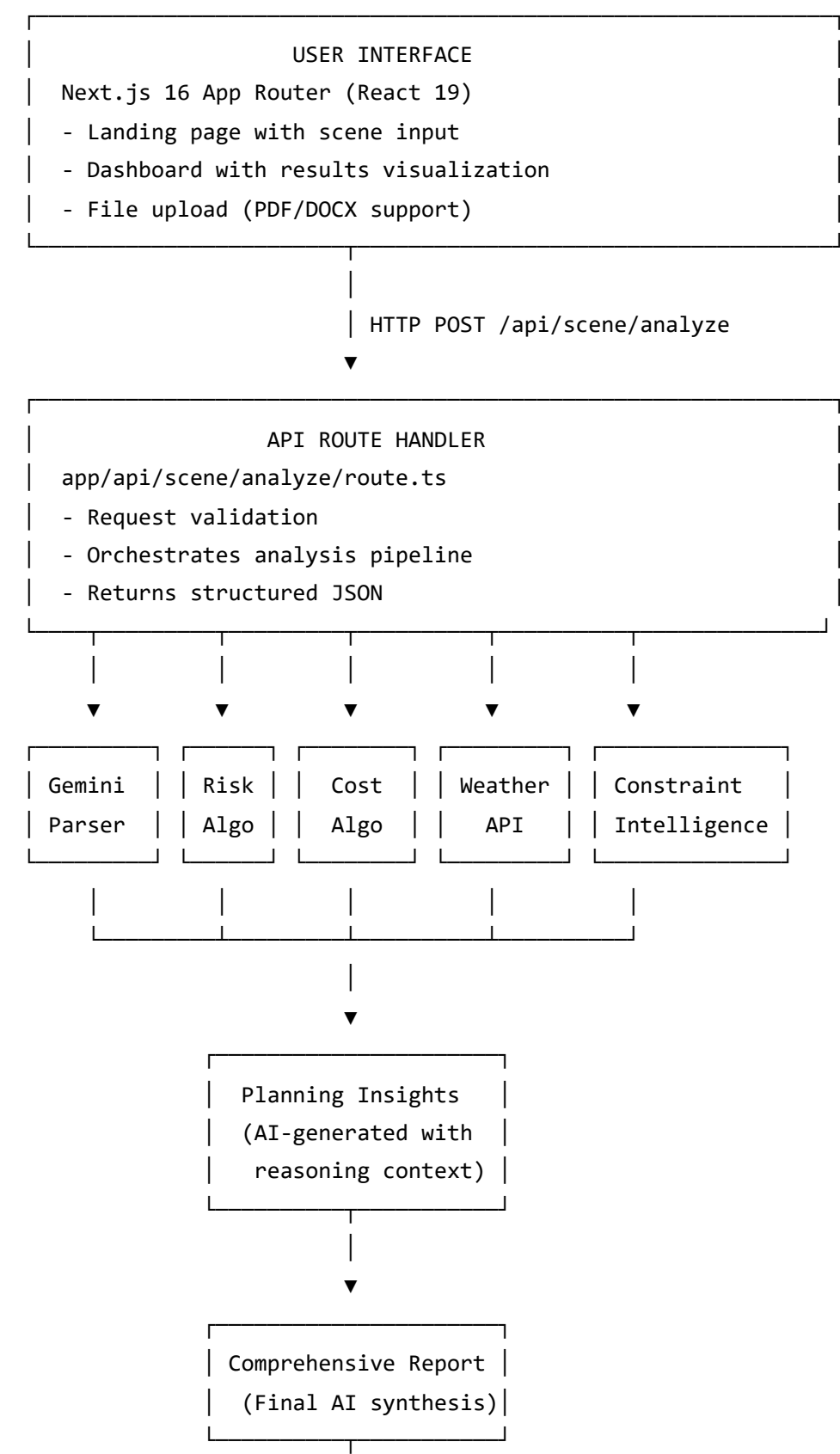
Logistics: High (Crowd) → Line item: Security, crowd wranglers
→ Permits: Public assembly permit
→ Budget: Extra background actors

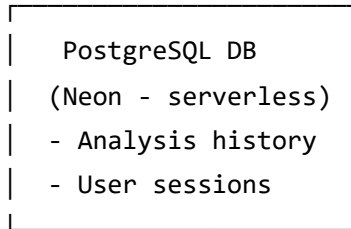
Technical: High (Night) → Line item: Lighting package rental
→ Crew: Gaffer, best boy electric
→ Schedule: Night differential rates

Weather Risk (12 rain days in June) → Schedule: Weather contingency days
→ Budget: Cover set rental
→ Insurance: Weather coverage

9. Technical Architecture (High Level)

System Components





LLM Usage Boundaries

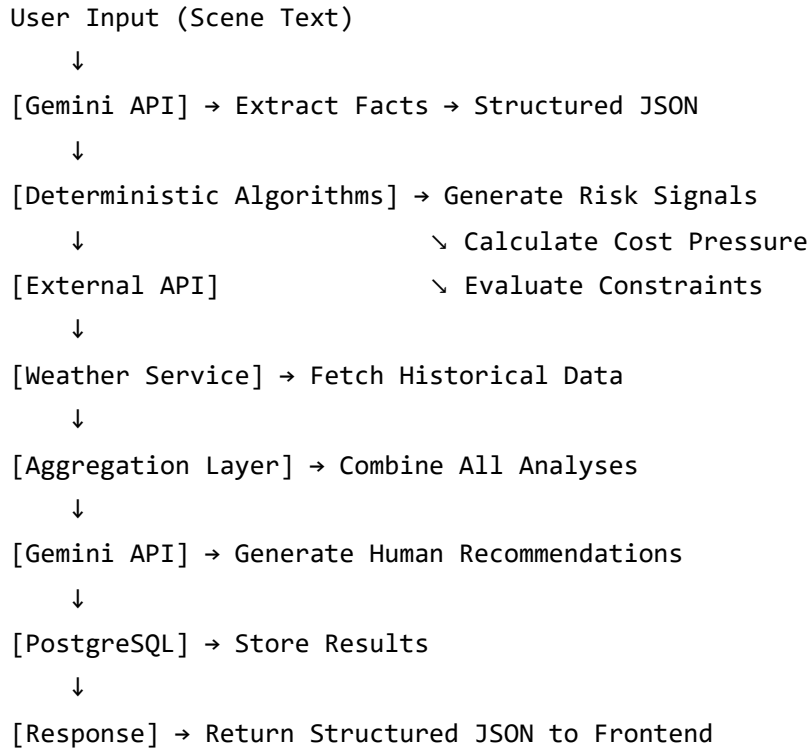
WHERE AI IS USED:

1. **Fact Extraction** (Gemini 2.0 Flash)
 - Input: Unstructured scene text
 - Output: Structured JSON with boolean flags and enum values
 - Temperature: 0.1 (deterministic)
2. **Planning Insights** (Gemini 2.0 Flash)
 - Input: Structured facts + risk signals
 - Output: Human-readable recommendations
 - Temperature: 0.4 (balanced)
3. **Comprehensive Analysis** (Gemini 2.5 Flash Lite)
 - Input: Complete analysis context
 - Output: Production readiness report
 - Temperature: 0.4 (balanced)

WHERE AI IS NOT USED:

- ☒ Risk signal generation (deterministic logic)
- ☒ Cost pressure calculation (algorithm-based)
- ☒ Constraint interaction evaluation (rule-based)
- ☒ Feasibility scoring (mathematical formula)

Data Flow



Key Design Decisions

Stateless API Design

- Each request is self-contained
- No server-side session state
- Scales horizontally with zero config

Database Persistence

- Drizzle ORM for type-safe queries
- PostgreSQL schema with indexed fields
- Analysis history for repeat comparisons

Error Handling

- Graceful degradation (see Section 10)
- Retry logic with exponential backoff for Gemini API
- Fallback responses when external services fail

10. Scalability & Reliability

Horizontal Scalability

Serverless Deployment Ready

- Next.js API routes are stateless → deploy on Vercel/AWS Lambda
- Each request handled independently → no shared state
- Database connection pooling via Neon → handles concurrent requests

Performance Characteristics

- Average analysis time: 5-8 seconds (dominated by LLM API calls)
- Database queries: <100ms (indexed on userId, timestamp)
- Weather API: ~500ms (cached when possible)

Concurrent User Support

Current Architecture Supports:

- 100+ concurrent analyses (Vercel/Neon free tier)
- 1000+ concurrent with Pro tier (same codebase)
- 10k+ concurrent with dedicated infrastructure (zero code changes)

Reliability & Graceful Degradation

Gemini API Failure:

```

// Retry logic with exponential backoff
let retries = 3
let delay = 2000 // 2 seconds

while (retries > 0) {
  try {
    return await gemini.generateContent(prompt)
  } catch (error) {
    if (error.code === 429) { // Rate limit
      await sleep(delay)
      delay *= 2
      retries--
    } else {
      throw error
    }
  }
}

// Fallback: Return minimal structured analysis
return {
  hasCrowd: false,
  hasStunts: false,
  actionIntensity: 'Medium', // Conservative default
  environmentComplexity: 'Medium'
}

```

Weather API Failure:

```

if (!weatherData) {
  return {
    applicable: true,
    recommendation: 'Weather data unavailable. Manual assessment recommended for outdoor scenes.'
  }
}

```

Database Failure:

```

// Analysis still returns to user (not saved to history)
// User gets results, but history page shows error

```

Data Persistence Strategy

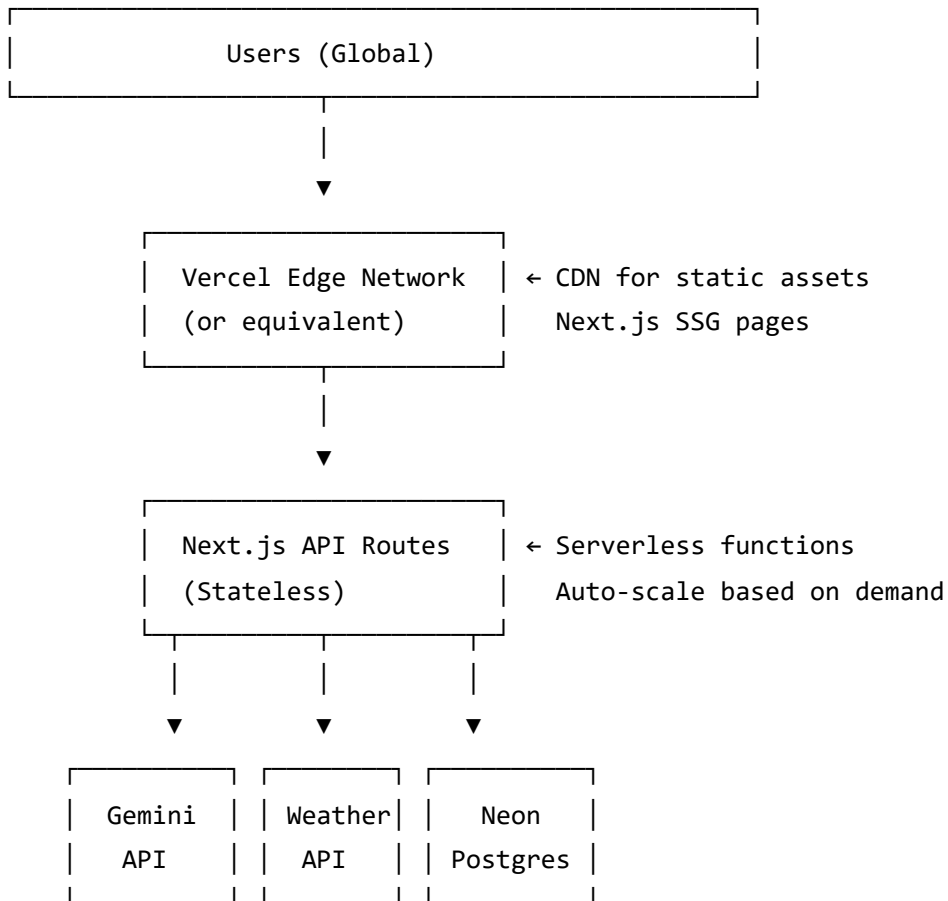
PostgreSQL Schema (Neon)

```
CREATE TABLE scene_analyses (  
  id UUID PRIMARY KEY,  
  user_id TEXT NOT NULL,  
  scene_description TEXT NOT NULL,  
  scene_category TEXT NOT NULL,  
  analysis_result JSONB NOT NULL, -- Full structured output  
  created_at TIMESTAMP DEFAULT NOW(),  
  INDEX idx_user_created (user_id, created_at)  
);
```

Why Neon/Postgres?

-  JSONB support for flexible schema
-  Serverless autoscaling (0 to thousands of connections)
-  Point-in-time recovery (data safety)
-  Standard SQL (easy to migrate if needed)

Production Deployment Architecture



Load Handling:

- Static pages served from CDN (instant)
- API routes auto-scale (Vercel handles container orchestration)
- Database pools connections automatically (Neon serverless driver)
- No manual infrastructure management needed

11. Why This Directly Solves the PS

Problem Statement Requirements ↔ SceneGuard Features

PS-3 Requirement	SceneGuard Implementation	Evidence
"Analyze a scene"	Scene text/file input → Multi-stage analysis pipeline	app/api/scene/analyze/route.ts orchestrates full pipeline
"Predict production feasibility"	Feasibility score (0-100) based on risk signals + constraints	lib/feasibility-score.ts calculates composite score
"Risk using reasoning-based evaluation"	Deterministic risk logic + constraint interaction analysis	lib/risk-analyzer.ts applies production rules to extracted facts
"High-risk, costly, or difficult to execute"	Evaluates Safety, Logistics, Budget, Technical dimensions separately	4 constraint categories with evidence mapping
"Budget constraints"	Cost pressure analysis based on scene complexity drivers	lib/cost-analyzer.ts identifies cost-intensive elements
"Logistics constraints"	Location complexity, crowd control, permits, access	Logistics risk signals + planning insights
"Safety constraints"	Stunt detection, hazard identification, safety crew needs	Safety risk category with specific mitigation steps
"Technical constraints"	VFX, lighting, equipment requirements	Technical risk signals + equipment checklists

Reasoning-Based Evaluation Proof

PS-3 explicitly requires "reasoning-based evaluation" — not just AI text generation.

SceneGuard achieves this through:

1. Fact Extraction ≠ Reasoning

- AI extracts `hasCrowd: true` (fact)
- Algorithm reasons: "Crowd + Outdoor → Logistics risk because permits + security needed" (reasoning)

2. Evidence-Grounded Constraints

- Every risk signal traces back to specific scene elements
- Constraint interactions are rule-based, not hallucinated
- Example: "Budget-Logistics interaction is High because remote location requires transport costs"

3. Deterministic Logic Layers

```
// This is reasoning, not generation
if (sceneCategory === 'Outdoor' && hasWeatherData) {
  if (averageRainDays > 10) {
    feasibilityPenalty += 10
    recommendations.push('High rain risk - plan weather cover')
  }
}
```

4. Reproducible Outputs

- Same scene → same risk score (proves algorithm-based reasoning)
- Different from ChatGPT/Gemini direct prompting (varies each time)

Direct PS Alignment

The problem statement says:

"Build an AI system that analyzes a scene and predicts production feasibility and risk using reasoning-based evaluation."

SceneGuard does exactly this:

-  AI system (uses Gemini for fact extraction)
-  Analyzes scenes (text + file input)
-  Predicts feasibility (0-100 score)
-  Predicts risk (Safety/Logistics/Budget/Technical categories)
-  **Uses reasoning-based evaluation** (deterministic algorithms reason over extracted facts)

Not just "AI-powered" — reasoning-enhanced AI.

12. What Makes SceneGuard Different

Comparison: SceneGuard vs. Direct Gemini Prompting

Dimension	Direct Gemini Prompt	SceneGuard
Output Structure	Freeform text, varies each time	Consistent JSON schema with typed fields
Reproducibility	Different answers on retry	Same scene → same risk score
Explainability	"Black box" recommendation	Each risk signal traces to specific scene element
Production Alignment	Generic film advice	Maps to real pre-production phases (crew, equipment, schedule)
Risk Granularity	Vague "high/low risk"	4 separate dimensions (Safety, Logistics, Budget, Technical)
Constraint Interaction	Not analyzed	Evaluates how constraints affect each other (e.g., Safety ↔ Logistics)
Weather Integration	Relies on user to check	Fetches real historical weather data for outdoor scenes
Cost Transparency	May claim exact budget	Honest "cost pressure" relative assessment
Reasoning Transparency	Opaque	Shows WHY each risk exists (evidence-based)
Database Persistence	No history	Stores all analyses for comparison
Production Checklist	Generic to-do list	Scene-specific requirements (e.g., "Hire stunt coordinator because scene has stunts")

Example: Same Scene, Different Analysis

Scene: "A crowded market chase scene at noon"

Direct Gemini Prompt:

Prompt: "Is this scene feasible?"

Response: "This scene is moderately feasible. You'll need to manage the crowd and ensure safety. Consider hiring a coordinator. Budget for extras and permits."

- ✗ Vague, no specifics
- ✗ No risk breakdown
- ✗ Can't compare to other scenes
- ✗ Different answer each time

SceneGuard Analysis:

```
{
  "feasibilityScore": 68,
  "riskSignals": [
    {
      "category": "Logistics",
      "level": "High",
      "reason": "Crowd control in public space requires permits, security, traffic management"
    },
    {
      "category": "Safety",
      "level": "Medium",
      "reason": "Chase scene requires safety protocols for background actors"
    }
  ],
  "productionChecklist": [
    "Secure filming permit for public market location",
    "Hire 2-3 security personnel for crowd control",
    "Schedule safety briefing for background actors",
    "Coordinate with local authorities for traffic control"
  ],
  "costDrivers": [
    "Background actor fees (crowd)",
    "Security personnel day rates",
    "Location permit fees"
  ]
}
```

- ✓ Structured and specific
- ✓ Reproducible (same each time)
- ✓ Actionable checklist
- ✓ Traces back to scene elements

13. Limitations & Honest Boundaries

SceneGuard intentionally does **NOT** claim to:

✗ Not a Budget Calculator

- **What it does:** Identifies cost pressure drivers (e.g., "Night shoot increases costs")
- **What it doesn't:** Predict exact dollar amounts (e.g., "\$45,000")
- **Why:** Budgets vary by region, union status, equipment rental rates, crew experience

✗ Not a Future Predictor

- **What it does:** Assess feasibility based on scene description
- **What it doesn't:** Guarantee production success
- **Why:** Real-world production depends on execution, team skill, unforeseen issues

✗ Not a Replacement for Human Producers

- **What it does:** Augment decision-making with systematic analysis
- **What it doesn't:** Make final go/no-go decisions
- **Why:** Producers bring context, relationships, creative judgment that AI cannot replace

✗ Not Real-Time Weather Forecasting

- **What it does:** Use historical weather data for planning
- **What it doesn't:** Predict weather on specific shoot dates
- **Why:** Weather API provides seasonal averages, not future forecasts

✗ Not Scene Editing or Rewriting

- **What it does:** Suggest mitigation strategies (e.g., "Consider studio instead of outdoor")
- **What it doesn't:** Rewrite scripts or change creative vision
- **Why:** Creative decisions remain with writers and directors

What SceneGuard IS

- ✓ **A decision-support system** that provides structured, reasoning-based feasibility analysis
- ✓ **A risk identification tool** that highlights Safety, Logistics, Budget, Technical constraints early
- ✓ **A production planning assistant** that generates scene-specific checklists and recommendations
- ✓ **A consistency layer** that ensures every scene gets the same rigorous evaluation

14. Future Scope

Logical extensions of the **current implemented system**:

1. Comparative Scene Analysis

Current: Analyze one scene at a time

Extension: Compare multiple scenes side-by-side

- Rank scenes by feasibility score
- Identify highest-risk scenes in script
- **Implementation:** Reuse existing pipeline, add comparison UI

2. Historical Analysis Trends

Current: Store analysis history in database

Extension: Show trends over time

- How many high-risk scenes analyzed this week?
- Common risk categories across projects
- **Implementation:** Aggregate queries on existing data

3. Custom Risk Weights

Current: Fixed risk scoring algorithm

Extension: Let users adjust weights

- Some productions prioritize safety over budget
- Others may weight technical complexity higher

- **Implementation:** Parameterize existing scoring formulas

4. Crew Recommendation Engine

Current: Generic "hire stunt coordinator" checklist

Extension: Specific crew role suggestions

- Based on scene complexity → recommend specific crew size
- Map risk categories → specific department heads needed
- **Implementation:** Extend planning insights logic

5. Budget Range Estimation

Current: Relative cost pressure (Low/Med/High)

Extension: Regional budget brackets

- Partner with production accountants for data
- Provide ranges (e.g., "10k—50k" for night shoot)
- **Implementation:** External data integration, keeping current pressure analysis

6. Multi-Language Scene Support

Current: English scene descriptions

Extension: Support for scripts in other languages

- Gemini supports 100+ languages
- Same reasoning pipeline works language-agnostically
- **Implementation:** Pre-processing translation layer

7. Storyboard/Image Analysis

Current: Text-based scene input

Extension: Analyze scene boards or concept art

- Use Gemini Vision to extract visual elements
- Same reasoning pipeline for risk evaluation
- **Implementation:** Vision API integration + existing logic

Conclusion

SceneGuard is a reasoning-based production feasibility system built for PS-3.

It solves the core problem: **How do you know if a scene is feasible before production?**

By combining:

- AI fact extraction (structured, not freeform)
- Deterministic risk reasoning (explainable, reproducible)
- Production-aligned analysis (Safety, Logistics, Budget, Technical)
- Evidence-grounded recommendations (traceable to scene elements)

SceneGuard is not "ChatGPT for films."

It's a systematic evaluation pipeline that ensures every scene gets the same rigorous, transparent, defensible feasibility assessment.

For judges evaluating PS-3 submissions:

This system directly addresses the problem statement's core requirement: **reasoning-based evaluation** of production feasibility. It's not AI magic — it's AI-assisted reasoning with deterministic logic, explainable outputs, and production-ready structure.

The code is production-grade, the architecture scales, and the approach is honest about what AI can and cannot do in film pre-production.

SceneGuard: From script to shoot, know before you go.

Repository: <https://github.com/Yaser-123/SceneGuard>

Tech Stack: Next.js 16, TypeScript, Gemini 2.0, PostgreSQL (Neon), Drizzle ORM

Deployment-Ready: Vercel-compatible, stateless API design, horizontal scalability